

CHM Platform Roadmap — Dev, Cognito & MediaHub

Prepared for: CHM leadership, CHT platform team, MediaHub engineering

Updated: June 16, 2026

Status: Approved direction — destroy staging first, Cognito prod, then stable dev (local + AWS), MediaHub microservices on CHT platform estate

Executive Summary

This plan consolidates architecture and sequencing decisions for the CHM platform:

1. **Destroy staging** — decommission AWS staging stack to reduce cost and complexity.
2. **Migrate authentication to Amazon Cognito on production** — **migrate existing GoTrue users**, do not create new accounts. Use **local Cognito** on developer machines during the build/cutover window.
3. **After Cognito prod is up and running**, stand up a **stable dev environment** (local docker-compose + optional shared AWS dev + dev Cognito pool mirroring prod config).
4. **Move MediaHub** (`mediahub-api`, `mediahub-worker`, `mediahub-reports`) into the CHT AWS footprint as **separate microservices** in the same ECS cluster — separate repo, separate deploys, separate RDS.
5. **CHT-only Redis cache** (24h TTL) with **catalog refresh on sync** — no version keys, no FastAPI middleware.

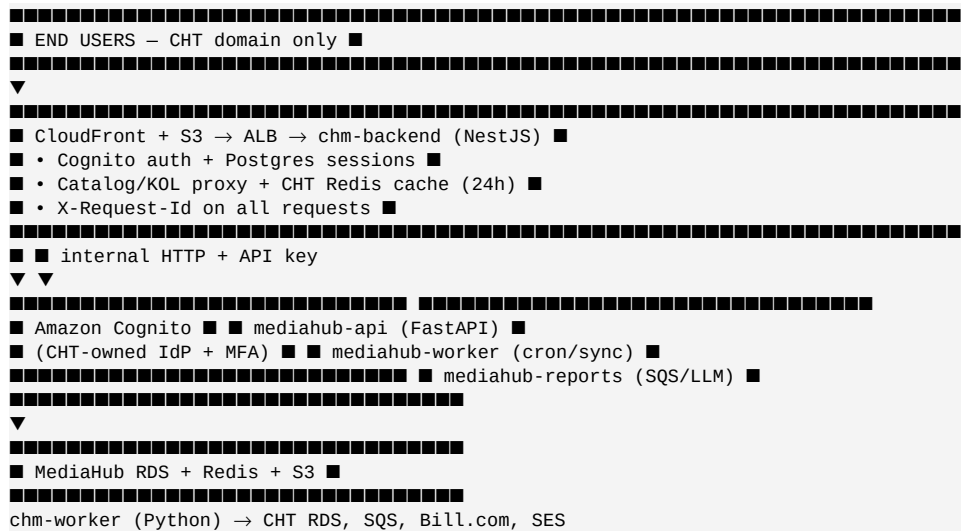
Total estimated timeline: 3–4 months.

Architecture Principles

Decision	Choice
Auth	CHT-owned Cognito + required TOTP MFA; end users never authenticate on MediaHub
Users	Migrate existing GoTrue users; remap <code>User.authId</code> in CHT Postgres — no greenfield accounts
Environments	Prod first (Cognito) , then stable dev + local — staging decommissioned
Local dev	docker-compose Postgres; local Cognito (emulator or dedicated dev pool) for auth testing
MediaHub	Separate <code>mediahub-platform</code> repo; independent ECS deploys

Decision	Choice
Cluster	One ECS cluster per environment: chm-backend, chm-worker, mediahub-api, mediahub-worker, mediahub-reports
MediaHub database	Separate RDS instance — Postgres 15.17, db.t3.small, Multi-AZ (prod)
CHT ↔ MediaHub	Internal HTTP + API key only; chm-backend never connects to MediaHub Postgres
Cache	CHT-only ElastiCache Redis; 24h TTL; worker clears catalog keys after successful sync
Language	Keep MediaHub in Python (FastAPI) — no TypeScript rewrite
Service mesh	Not required — security groups + internal DNS; ECS Service Connect optional later

End-State Architecture



ECS services (production)

Service	Tasks (typical)	Database
chm-backend	2–3	CHT RDS
chm-worker	1–2	CHT RDS

Service	Tasks (typical)	Database
mediahub-api	2	MediaHub RDS
mediahub-worker	1	MediaHub RDS
mediahub-reports	1 (+ SQS scale)	MediaHub RDS

MediaHub RDS (production)

Setting	Value
Engine	PostgreSQL 15.17
Instance	db.t3.small
Storage	50 GB gp3 (grow as needed)
Multi-AZ	Yes
Backup retention	30 days
Access	mediahub-api, mediahub-worker, mediahub-reports SGs only

CHT-Only Cache Strategy

Scope: Cache public catalog and KOL network reads on chm-backend only. No HTTP cache in MediaHub. No FastAPI middleware.

TTL: 86400 seconds (24 hours) as safety net.

Refresh on sync: When mediahub-worker completes a successful sync, call CHT internal endpoint to clear catalog cache keys. Next user request repopulates from MediaHub with new content.

```
Normal: User → chm-backend → Redis hit → return (up to 24h)
Miss: User → chm-backend → mediahub-api → store in Redis EX 86400
After sync:
mediahub-worker → POST /internal/cache/catalog/clear
chm-backend → DEL cht:catalog:* and cht:kol-network:*
Next request → cache miss → fresh data → cache 24h again
```

Redis key patterns:

Key	Cached
cht:catalog:tags	Yes

Key	Cached
cht:catalog:clips:{hash(params)}	Yes
cht:catalog:playlists:{hash}	Yes
cht:kol-network:{hash}	Yes
Auth, payments, admin, writes	Never

Infrastructure: One ElastiCache cluster (cache.t3.micro or small) with key prefixes; chm-backend reads/writes catalog keys; worker triggers clear via internal secret + SG-restricted endpoint.

Tracing (Request ID)

Phase in during MediaHub migration (Phase 5):

- Generate or preserve `X-Request-Id` at chm-backend (ALB or NestJS middleware).
 - Forward to mediahub-api on every internal call.
 - Log JSON with `request_id` in CloudWatch on both sides.
 - Pass `request_id` in SQS message attributes for report jobs.
 - Optional later: OpenTelemetry → AWS X-Ray (not App Mesh).
-

Phase 0 — Spec & Runbooks (1–2 weeks)

Goal: Signed documentation before infra changes.

Deliverables:

- Cognito Migration & MediaHub Auth Decommission specification
- User migration runbook: GoTrue export → Cognito import → authId remap (no new users)
- Staging teardown checklist
- Local Cognito setup guide (developer machines during Phase 2)
- Stable dev environment specification (for Phase 3 — after Cognito prod)
- MediaHub cutover runbook: EC2 DB → new RDS, internal URL
- Cache & sync contract: Redis keys, internal clear endpoint, worker hook

Exit criteria: Runbooks approved; rollback defined for each phase.

Phase 1 — Destroy Staging (1 week)

Goal: Decommission staging AWS stack before Cognito work. No replacement dev env required yet.

Prerequisites:

- Document any staging secrets needed for prod (copy to prod GitHub environment)
- Confirm no production dependency on staging.testapp.communityhealth.media
- Optional final snapshot of staging RDS if anything worth retaining

Steps:

- `terraform destroy` on us-east-1-staging (ECS, RDS, ALB, SQS, CloudFront)
- Retire `deploy-staging.yml` or archive workflow
- Remove GitHub staging environment (after secrets documented)
- Update `deployment.md`, `incident-response.md`, `architecture.md`
- Retire DNS: staging.testapp.communityhealth.media

During Phase 1–2 (no AWS dev yet):

- Local development: `docker-compose` Postgres + backend/frontend/worker via `.env`
- Auth testing during Cognito build: **local Cognito** (see Phase 2)

Exit criteria:

- Staging AWS resources destroyed; billing stopped
 - Team aligned on prod + local-only until Phase 3
-

Phase 2 — Cognito on Production (4–7 weeks)

Goal: Production auth on Cognito; existing users migrated; GoTrue decommissioned for end users.

2a — Local Cognito for development (parallel with build)

Use **local Cognito on developer machines** while implementing auth — no AWS dev env required yet.

Option	Use case
AWS Cognito dev pool (small, cheap)	Closest to prod behavior; shared by team
cognito-local / LocalStack Cognito	Fully offline auth flow testing on laptop
VITE_USE_DEV_AUTH=true	Bypass auth for non-auth feature work only

Document in repo: pool IDs, client IDs, redirect URLs for localhost.

Local stack during Phase 2:

```
localhost:5173 (frontend) → localhost:3000 (backend) → local Postgres
↓
local/dev Cognito pool
(NOT MediaHub GoTrue)
```

2b — Infra (week 1–2)

- Terraform: Cognito **prod** user pool, app client `cht-web`
- Cognito groups: `cht-kol`, `cht-hcp`, `cht-pharma-client`
- MFA: required TOTP (define grace period for enrollment)
- App client `mediahub-admin` for CHM operators (MediaHub UI)
- Secrets Manager: Cognito pool ID, client IDs; retain GoTrue secrets until cutover day

2c — Application (week 2–5)

- Backend: Cognito `SignUp`, `InitiateAuth`, MFA challenge, JWKS validation
- Remove GoTrue proxy from `auth.controller.ts`
- Frontend: Cognito on CHT domain only; remove MediaHub OAuth redirects
- Keep Postgres sessions + `httpOnly` cookies
- Chatbot: Cognito token via backend
- **Unchanged:** `MediaHubService`, `MediaHubSyncService` (API key only)

Validate on local Cognito + local docker before prod cutover.

2d — User migration — no new users (week 5–6)

1. Export GoTrue users: email, sub, `user_metadata` (NPI, profession, institution).
2. Import into Cognito prod via bulk import or User Migration Lambda.
3. Remap CHT Postgres `User.authId`: GoTrue sub → Cognito sub (match by email).
4. OAuth users: federate Google in Cognito; link by email to existing User row.
5. Communicate: same account, one-time MFA setup — not a new registration.

Pre-cutover validation:

- Auth flows proven on **local Cognito** + local backend
- Dry-run import counts match GoTrue export
- Dry-run `authId` remap on prod DB snapshot (read-only)
- Rollback plan: keep GoTrue secrets 48h post-cutover

2e — MediaHub auth coordination (parallel)

- Block end-user OAuth redirects to MediaHub
- Deactivate non-admin GoTrue users (learners/HCP/KOL) — content data unchanged
- Confirm API key valid through and after cutover

2f — Production cutover (week 6–7)

- Maintenance window + user comms
- Import users to Cognito prod; run authId remap
- Deploy Cognito-enabled backend + frontend
- MediaHub: GoTrue off for end users
- Monitor login, MFA, catalog, HCP upsert, payments
- Remove GOTRUE_JWT_SECRET, SUPABASE_* from prod secrets (after 48h rollback window)
- Update subprocessor register

Exit criteria:

- Cognito prod **up and running**; existing users log in with same email
- No duplicate User rows created
- End users never touch MediaHub auth
- Catalog still works via API key (external MediaHub URL until Phase 4)

Phase 3 — Stable Dev Environment (2–3 weeks)

Goal: After Cognito prod is stable, create a durable non-prod environment for ongoing development.

Prerequisites:

- Cognito prod cutover complete and stable ($\geq$ 1–2 weeks monitoring)
- Staging already destroyed (Phase 1)

3a — Local dev (document and standardize)

Layer	Setup
Database	docker-compose Postgres (existing)
Backend / frontend / worker	.env + npm/pip local run
Auth	Local Cognito — dedicated AWS dev pool OR cognito-local; mirrors prod pool config
MediaHub	External MediaHub API URL + dev API key (until Phase 4)
Dev auth bypass	VITE_USE_DEV_AUTH=true for UI-only work (optional)

3b — Shared AWS dev (optional, week 2–3)

Small stack mirroring prod topology — only if team needs shared integration testing:

- Dev Cognito user pool (separate from prod; clone prod app client settings)
- Optional: small ECS (1 task each) or deploy-on-demand to dev

- db.t3.micro RDS for CHT dev data — **never prod credentials**
- GitHub dev environment + deploy workflow

Dev Cognito pools:

Pool	Purpose
Prod pool	Production users (Phase 2)
Dev pool	Engineering test accounts (Phase 3)
Local Cognito	Laptop-only auth during implementation (Phase 2+)

3c — Documentation

- README: local setup, local Cognito bootstrap, env var matrix
- Prod deploy runbook (no staging gate — use checklist + local validation)

Exit criteria:

- New engineers can run full stack locally with local/dev Cognito
- Optional AWS dev available for integration tests
- Prod remains the only customer-facing environment

Phase 4 — MediaHub Platform Move (6–10 weeks)

Goal: mediahub-api, mediahub-worker, mediahub-reports on CHT ECS cluster; separate RDS; internal integration.

4a — Repository & CI (week 1–2)

- Create mediahub-platform repo (Python / FastAPI)
- Separate Dockerfiles and deploy workflows per service
- Publish OpenAPI for /api/public/*
- Contract tests consumable by CHT CI

4b — Production infra (week 2–4)

Add to us-east-1 Terraform:

- RDS module: mediahub-db (Postgres 15.17, db.t3.small, Multi-AZ, 50 GB)
- ElastiCache for CHT catalog cache (+ optional Hub locks)
- ECS: mediahub-api (×2), mediahub-worker (×1), mediahub-reports (×1)
- SQS report queue + DLQ
- S3 buckets for reports/assets

- Secrets: mediahub/database, API keys, LLM keys
- Security groups: Hub DB accessible only from Hub services

4c — Data migration (week 4–5)

- pg_dump from MediaHub EC2 Postgres
- Restore to mediahub-db
- Validate clips, KOLs, tags, analytics row counts
- Point mediahub-api at new RDS
- No new auth users — identity already on Cognito from Phase 2

4d — Cutover (week 5–6)

- Deploy Hub services to prod ECS (same cluster as chm-backend)
- Update CHT MEDIAHUB_BASE_URL to internal service URL
- Integration tests: catalog, KOL network, HCP upsert
- Decommission MediaHub EC2 Compose stack
- Optional: admin hostname to new mediahub-api

4e — CHT cache + sync refresh (week 6–7)

- Wire ElastiCache to chm-backend
- NestJS cache on catalog/KOL routes (24h TTL)
- Implement POST /internal/cache/catalog/clear
- mediahub-worker calls clear after successful sync
- Verify new clips appear on next request after sync

Exit criteria:

- Five ECS services deploy independently
- EC2 MediaHub retired
- CHT calls Hub internally; API key auth unchanged
- Cache refreshes on sync

Phase 5 — Hardening (ongoing)

- X-Request-Id across chm-backend ↔ mediahub-api ↔ SQS jobs
- CloudWatch alarms: Hub ECS, both RDS instances, Redis, DLQs
- DR drill: restore mediahub-db from snapshot
- Performance Insights: upgrade MediaHub RDS if CPU sustained > 60%
- Auth + catalog contract tests in CI
- Optional: OpenTelemetry / X-Ray

Dependency Order

```
Phase 0: Specs & runbooks
↓
Phase 1: Destroy staging
↓
Phase 2: Cognito prod (local Cognito for build/validation)
↓
Phase 3: Stable dev environment (after Cognito prod is running)
↓
Phase 4: MediaHub ECS + separate RDS + cache
↓
Phase 5: Hardening
```

Destroy staging before Cognito — no dev env blocker.

Stand up stable dev only after Cognito prod is up and stable.

Use local Cognito on developer machines during Phase 2 implementation.

Timeline Overview

Phase	Duration	Outcome
0 — Spec	1–2 weeks	Signed runbooks
1 — Destroy staging	~1 week	Staging gone; cost saved
2 — Cognito prod	4–7 weeks	Prod auth migrated; local Cognito used during build
3 — Stable dev	2–3 weeks	Local + optional AWS dev; dev Cognito pool
4 — MediaHub move	6–10 weeks	Microservices on platform; internal URL
5 — Hardening	Ongoing	Tracing, DR, monitoring

Total: approximately 3–4 months.

Cost Estimate

Change	Approx. monthly impact
Destroy staging	Save ~\$50–100
Cognito prod	~\$0–5 (MAU-based)
AWS dev (Phase 3, optional)	~\$40–80
MediaHub RDS (db.t3.small Multi-AZ)	+\$55–75
ElastiCache (cache.t3.micro)	+\$12–25
MediaHub Fargate (api×2, worker×1, reports×1)	+~\$72
LLM API (reports)	Variable

Risks & Mitigations

Risk	Mitigation
No staging; Cognito before dev env	Local Cognito + local docker for Phase 2; strict prod cutover checklist
No AWS dev until Phase 3	Accept prod-only customer env; local stack for daily dev
User migration mismatch	Email-based remap; reconciliation report before cutover
MFA adoption friction	Clear comms: same account, MFA enrollment once
MediaHub cutover breaks catalog	Parallel run EC2 + ECS until validated; API key unchanged
Cache stale after sync	Worker clears catalog keys; 24h TTL as backstop

Immediate Next Actions

1. Sign off Phase 0 runbooks (staging teardown, Cognito user migration, local Cognito guide).
2. **Execute Phase 1:** terraform destroy staging; document migrated secrets.
3. Open Terraform PR for Cognito prod pool + local/dev pool for laptop testing.
4. Document local Cognito bootstrap in getting-started.md.
5. Confirm MediaHub liaison: GoTrue shutdown date + API key continuity.

Related Documents

- [CHT-MediaHub-Go-Forward-Options.md](#)
- [CHT-Auth-Decoupling-Next-Steps-Report.md](#)
- [CHT-Platform-Assessment-Report.md](#)

Prepared by: CHT Engineering (roadmap consolidated June 16, 2026)